Lab_4_GCN

November 10, 2025

0.1 GNN on connectome

We will go through a tutorial with toy data

and then a link for EEG data with explainability (optional)

Nevertheless, this is the standard 10-20 settings for the electrode:

0.2 Understanding GCN concept by building a simple graph . . .

0.2.1 Defining the graph and it's features

So, we build the adjacency matrix A from the **LEFT** figure (graph woth 5 nodes):

[1]:
```
import numpy as np
##############################

##############################
A = np.matrix([
    [0, 1, 1, 1, 0],
    [1, 0, 1, 0, 1],
    [1, 1, 0, 0, 1],
    [1, 0, 0, 0, 0],
    [0, 1, 1, 0, 0]
    ],dtype=float
)
```

Now, we generate 2 features for each node based on the index. This makes it easy to confirm the matrix calculations manually later.

[2]:
```
X = np.matrix([
            [i, -i]
            for i in range(A.shape[0])
        ], dtype=float)
X
```

[2]:
```
matrix([[ 0.,  0.],
        [ 1., -1.],
        [ 2., -2.],
```

```
         [ 3., -3.],
         [ 4., -4.]])
```

[3]:
```
A*X
```

[3]:
```
matrix([[ 6., -6.],
        [ 6., -6.],
        [ 5., -5.],
        [ 0.,  0.],
        [ 3., -3.]])
```

0.2.2 Applying the simple propagation rule for the input layer:

Now we will try to implement the layer equation defined in theory. To make it easier, we will ommit the normalization part, as it is implicit in the implementation of the model from Pytorch Geometric that we will use and it is not necessary to get the intuition of how GCN works.

0.2.3 f(H , A) = (A H W)

- H = X = input features
- AH W = AXW = AX
- W = I

[4]:
```
H_0 = X
```

[5]:
```
W = np.identity(2)
```

[6]:
```
H_1 = A * H_0 * W
H_1
```

[6]:
```
matrix([[ 6., -6.],
        [ 6., -6.],
        [ 5., -5.],
        [ 0.,  0.],
        [ 3., -3.]])
```

As you can observe, the representation of each node (each row) is now a sum of its neighbors features!

In other words, the graph convolutional layer represents each node as an aggregate of its neighborhood, as we saw on theory slides.

0.2.4 Adding self loops

If you look carefuly to the previous representation of each node after the popagation rule, the node itself is not taken into account. Thus, that is the reason why we need to add self loops to the adjacency matrix, conforming the Â matrix.

[7]:
```
############################
#
############################

I  = np.identity(A.shape[0])
I
```

[7]:
```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

[8]:
```
A_hat = A + I
A_hat
```

[8]:
```
matrix([[1., 1., 1., 1., 0.],
        [1., 1., 1., 0., 1.],
        [1., 1., 1., 0., 1.],
        [1., 0., 0., 1., 0.],
        [0., 1., 1., 0., 1.]])
```

[9]:
```
# Now run again the GCN equation and observe the new output
H_1 = A_hat * H_0 * W
H_1
```

[9]:
```
matrix([[ 6., -6.],
        [ 7., -7.],
        [ 7., -7.],
        [ 3., -3.],
        [ 7., -7.]])
```

As you observe, now we have taken into account the node itself when computing the aggregation of local neightbours by A * X.

0.2.5 Why do we need to normalize?

Here we intent to show what happens without normalizing. Then, we will show whats the same output when applying normalization.

So, lets try to apply the GCN equation for 1000 epochs:

[10]:
```
H_n = H_1
for i in range(1000):
    H_n = A_hat * H_n * W

print(H_n)
```

```
[[nan nan]
 [nan nan]
```

```
[nan nan]
[nan nan]
[nan nan]]
```

You can observe that the output features for epoch 1000 are nan... So, let's try to normalize before and repeat the operation.

Q: WHY DO YOU THINK IT HAPPENS?

The NaN values occur due to numerical instability from exploding gradients. When inputs aren't normalized, large values get multiplied through the network's weights during forward propagation, producing extremely large numbers that exceed floating-point limits. During backpropagation, gradients become too large, causing weight updates to overshoot and produce NaN (Not a Number) values.

[11]:
```
A_hat
```

[11]:
```
matrix([[1., 1., 1., 1., 0.],
        [1., 1., 1., 0., 1.],
        [1., 1., 1., 0., 1.],
        [1., 0., 0., 1., 0.],
        [0., 1., 1., 0., 1.]])
```

[12]:
```
D = np.array(A_hat.sum(1))
D
```

[12]:
```
array([[4.],
       [4.],
       [4.],
       [2.],
       [3.]])
```

[13]:
```
D_inv_half = np.power(D, -0.5).flatten()
D_mat = np.diag(D_inv_half)
# SYMMETRIC NORMALIZATION = D^-1/2 * (H) * D^-1/2
aux = D_mat.dot(A_hat)
norm_A_hat = aux.dot(D_mat)
norm_A_hat
```

[13]:
```
matrix([[0.25      , 0.25      , 0.25      , 0.35355339, 0.        ],
        [0.25      , 0.25      , 0.25      , 0.        , 0.28867513],
        [0.25      , 0.25      , 0.25      , 0.        , 0.28867513],
        [0.35355339, 0.        , 0.        , 0.5       , 0.        ],
        [0.        , 0.28867513, 0.28867513, 0.        , 0.33333333]])
```

[14]:
```
H_1_norm = norm_A_hat * H_0 * W
H_1_norm
```

[14]:
```
matrix([[ 1.81066017, -1.81066017],
        [ 1.90470054, -1.90470054],
        [ 1.90470054, -1.90470054],
        [ 1.5       , -1.5       ],
        [ 2.19935874, -2.19935874]])
```

[15]:
```
H_n = H_1_norm
for i in range(1000):
    H_n = norm_A_hat * H_n * W

print(H_n)
```

```
[[ 2.02009928 -2.02009928]
 [ 2.02009928 -2.02009928]
 [ 2.02009928 -2.02009928]
 [ 1.4284259  -1.4284259 ]
 [ 1.7494573  -1.7494573 ]]
```

Now, as you observe we still can get good results after iterating for 1000 epochs.

The explanation is that normalizing avoid numbers to be higher than one in all the multiplications, and so we need to do it in order to solve possible problems such as exploding gradients or having weights which are too big.

Below there is an example of how you can do this with Pytorch Geometric

[16]:
```
import torch

# Normalize the adjacency matrices
def normalize_adj(A):
  # Fill diagonal with one (i.e., add self-edge)
  A_mod = A + torch.eye(A.shape[1])

  # Create degree matrix for each graph
  diag = torch.sum(A_mod, axis=1)
  D = torch.diag(diag)

  # Create the normalizing matrix
  # (i.e., the inverse square root of the degree matrix)
  D_mod = torch.linalg.inv(torch.sqrt(D))

  # Create the normalized adjacency matrix
  A_hat = torch.matmul(D_mod, torch.matmul(A_mod, D_mod))
  A_hat = torch.tensor(A_hat, dtype=torch.float64)
  return A_hat


class GCNLayer(torch.nn.Module):
  def __init__(self, dim, hidden_dim):
```

```
    super(GCNLayer, self).__init__()
    self.W = torch.zeros(
      dim,
      hidden_dim,
      requires_grad=True,
      dtype=torch.float64
    )
    torch.nn.init.xavier_uniform_(self.W, gain=1.0)

  def forward(self, A_hat, h):
    # Aggregate
    h = torch.matmul(A_hat, h)

    # Update
    h = torch.matmul(h, self.W)
    h = F.relu(h)
    return h


class GCN(torch.nn.Module):
  def __init__(self, x_dim, hidden_dim1, hidden_dim2, hidden_dim3):
    super(GCN, self).__init__()

    # Convolutional layers
    self.layer1 = GCNLayer(x_dim, hidden_dim1)
    self.layer2 = GCNLayer(hidden_dim1, hidden_dim2)
    self.layer3 = GCNLayer(hidden_dim2, hidden_dim3)
    # Output layer linear layer
    self.linear = torch.nn.Linear(hidden_dim3, 1, dtype=torch.float64)

  def forward(self, A_hat, x):
    # Aggregate
    #x = torch.matmul(A_hat, x)

    # GCN layers
    x = self.layer1(A_hat, x)
    x = self.layer2(A_hat, x)
    x = self.layer3(A_hat, x)

    # Global average pooling
    x = torch.mean(x, axis=0)
    x = self.linear(x)
    return F.sigmoid(x)

  def parameters(self):
    params = self.layer1.parameters() \
      + self.layer2.parameters() \
```

```
            + self.layer3.parameters() \
            + list(self.linear.parameters())

        return params
```

0.3 Let's use the GNN with brain connectivity data (OPTIONAL)

We can use it to classify fucntional connectivity data from EEG or fMRI. For example from schizophrenia or people experiencing epileptic seizures.

Here is a public dataset with EEG and epileptic seizures recording: https://physionet.org/content/chbmit/1.0.0/

Use the following code based geometric pytorch for the GNNs and GATv2Lightning for explainability: https://github.com/szmazurek/sano_eeg

0.4 Why do we need Laplacian normalization in GCNs?

The Laplacian normalization in GCN's message passing serves two key purposes:

Prevents scale explosion: Without normalization, nodes with high degree would accumulate much larger feature values than low-degree nodes, leading to numerical instability and vanishing/exploding gradients.

Balances neighbor contributions: The symmetric normalization $D^(-1/2)AD^(-1/2)$ ensures that messages from high-degree nodes don't dominate and that high-degree nodes don't get overwhelmed by aggregating too many features. Each edge contributes proportionally to $1/\sqrt{(deg(i) \cdot deg(j))}$.

In essence, it makes the aggregation degree-invariant and stable across the graph structure.